

Final Report: Clinical Agent - A simple version of Agent Hospital

Philippe Schoeb

Mila

University of Montreal

philippe.schoeb@umontreal.ca

Nhung Dang

Rali

University of Montreal

thi.hong.nhung.dang@umontreal.ca

Abstract

Healthcare systems are essential parts of human society, yet they are under a lot of pressure and this can have disastrous effects on the population. Large Language Model (LLM) medical agents are recent technological tools that can help doctors, nurses and patients to reduce this pressure in many ways. A recent LLM training paradigm, proposed by **Agent Hospital** (Li et al., 2025), consists of a simulated hospital environment and has demonstrated outstanding results by implementing medical agents that outperform state-of-the-art methods on a prestigious medical test, **MedQA** (Jin et al., 2020). To pursue the advancements of doctor agents, it is crucial to analyze how their proposed methodology affects models with fewer resources. Hence, our project focuses on the following question: Can Agent Hospital’s methodology be used to enhance lower-scale doctor agents? To answer this question, we have implemented **Clinical Agent**¹ from scratch, a two step pipeline that includes patient generation and simulation of patient-doctor interactions. Then, we have evaluated the agents on MedQA. Down the line, we have seen an occurrence of performance enhancement using the simulation training. Indeed, Doctor Agent 1.0 has reached a test accuracy of **0.3284** on MedQA (with LoRA) which outperforms the baselines and every other model when using the same evaluation strategy. However, our results are not consistent enough to ensure improvement of low-resource doctor agents within this framework. Our main takeaway is that further efforts are needed in model adaptation, data realism, and simulation design to bridge the gap between synthetic simulation and real-world medical reasoning for modest LLMs.

1 Introduction

Virtual agents powered by LLMs are starting to see the day in the medical field to assist health

experts and answer basic patient questions. This support is highly valuable, so there have been recent efforts to improve these medical agents. Namely, **Agent Hospital** proposes a simulacrum of hospital in which doctor agents can evolve until reaching peak performance that outclasses other methods when evaluated on the **MedQA** (Jin et al., 2020), a multiple choice question answering task that requires medical expertise. Now, we have decided to contribute to the improvement of LLM doctor agents by analyzing whether or not this newly published technique can also improve the capabilities of LLM doctor agents with limited resources compared to regular fine-tuning. This kind of analysis on a lower scale is essential when it comes to down to understanding and optimizing popular training techniques.

Our contributions to this project consist of building the complete clinical agent processing pipeline and analyzing the performance of the resulting models. That pipeline is composed of a generation process that has been tested with several LLMs (Mistral_7B, Llama2_7B, Llama3_8B) and that has been evaluated using many techniques (expert evaluation, cosine similarity, BERTScore) for patient filtering. It also includes the simulation and evaluation of the doctor agents which have taken many forms (full fine-tuning, partial fine-tuning, LoRA) and that led to 5 different versions of doctor agents. This evaluation procedure has two parts and these were shown in Agent Hospital: real-life evaluation that corresponds to testing the agent on MedQA and virtual evaluation that corresponds to the performance of the agent on the simulation task.

For our first attempt, we used full fine-tuning to evaluate Doctor Agent 1.0 on MedQA and it was not successful since the best accuracies (0.3357 and 0.3543 for validation and test accuracies respectively) were obtained by the baselines we had

¹**Clinical Agent** https://github.com/PhilSchoeb/clinical_agent.git

built. Following that, we used LoRA instead and the best accuracies (0.3129 and 0.3284 for the respective metrics again) were reached by the PMB baseline and Doctor Agent 1.0 respectively. This is promising, but the 4 other doctor agents underperform which displays the inconsistency of our training method with modest LLMs. Some pretreatment strategies were used during patient generation. Specifically, the creation and usage of *augmented patients* had a negative impact on the doctor agent’s performance and patient filtering had a positive impact on the model’s outcome. All in all, we have come to the conclusion that we cannot claim the effectiveness of the simulation on low-scale doctor agents for now. In addition, further work would be needed to claim the ineffectiveness of this strategy in our given context.

2 Related work

Agent Hospital: A Simulacrum of Hospital with Evolvable Medical Agents (Li et al., 2025) serves as foundation of our project. It demonstrates that training a SoTA LLM via hospital simulation improves its performance compared to using other training techniques. An interesting parallel can be made with actual doctors, that learn most of their baggage from experiences on the terrain rather than from studies or textbooks. However, this paper is recent (January 17, 2025), so it lacks source code and uses a Chinese only dataset **Baidu Health Encyclopedia**², limiting direct reuse. Thus, we developed our approach from scratch, integrating course concepts. Moreover, the paper does not explicitly explain how the learning of doctor agents was implemented during the simulation. Consequently, we apply fine-tuning on a classification task to improve our doctor agent since this method is known to work efficiently.

BERTScore: Evaluating Text Generation with BERT (Zhang et al., 2020) proposes a token-level evaluation method based on contextual embeddings. It offers fine-grained metrics (precision, recall, F1) and open-source³. It can be installed by `pip install bert-score`. Although, it was mainly validated on machine translation and image captioning tasks, its flexibility for other generation types, like ours for text

²Baidu Health Encyclopedia <https://jianshang.baidu.com/widescreen/entitylist>

³BERTScore https://github.com/Tiiiger/bert_score

generation, remains unproven. To mitigate this, we combine **BERTScore** with *cosinus similarity* to ensure a more robust and reliable evaluation of our generated patient profiles.

3 Methods

3.1 Overview of the Clinical Agent Processing Pipeline

Before presenting the details, it is helpful to first understand the overall structure of the method. It consists of two asynchronous steps:

1. The **generation** and **modification** of patients.
2. The **training** of the doctor agent using the patients generated earlier and the textbook from MedQA (?).

In this work, we focus on generating patients from two departments: *Cardiology* and *Infectious and Parasitic Diseases*. The general pipeline is as follows:

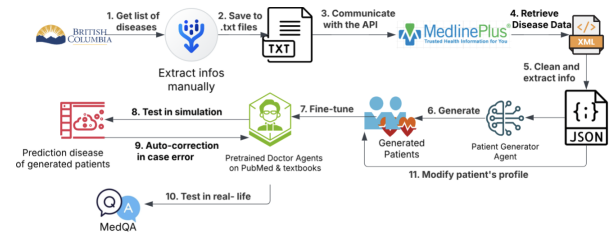


Figure 1: Overview of the clinical agent processing pipeline: from disease data extraction to simulated and real-life testing with doctor agents.

3.2 Synthetic Patient Data Pipeline

3.2.1 Medical Data Retrieval and Preprocessing

Step 1 to 5 in Figure 1 illustrate this process. A list of diseases was retrieved from the *British Columbia government website* (Section 5.1) in PDF format to identify common diseases in the two departments. An expert was consulted to validate the relevant diseases, which were saved in .txt files. The details for each disease were then retrieved using the *MedlinePlus API* (Section 5.1), with the returned data in XML format. The returned data are in XML format, reflecting the information displayed on the website, which limits their direct use. They were cleaned, and the relevant information was extracted and saved in JSON files (Section 5.1). Each disease entry in the JSON file contains the following fields:

- *title*: Main disease name.

- *altTitle*: Alternative disease name.
- *FullSummary*: Description with causes, symptoms, treatments.
- *mesh*: Disease category (MeSH).
- *groupName*: Broader disease group.
- *cause_diagnosis*: Example cause or diagnosis.

A short example of this json is provided in Annex B).

3.2.2 Synthetic Patient Profile Generation

Refer to steps 5 and 6 shown in the Figure 1. The JSON files created in the previous step were fed to an agent generator to create synthetic patients corresponding to each disease. At this stage, we generated synthetic patients using targeted prompts to interact locally with three language models acting as agents:

- *Mistral 7B* (Albert Q. Jiang, 2023)
- *LLama2 7B* (Hugo Touvron, 2023)
- *Llama3 8B - Instruct* (Llama Team, 2024)

3.2.3 Augmented Synthetic Patient Profiles

This part corresponds to Step 11 in Figure 1. To increase the difficulty of the simulation task for training the doctor agent, we introduce an alternative name for the correct disease into the patient's profile. Let us call these patients: *augmented patients*. The doctor agent must still correctly identify the masked disease despite this change.

3.3 Doctor Agent Training

3.3.1 Doctor Agents Baseline

The first two baselines used were the pre-trained language models **PubMedBERT** (Gu et al., 2021) that is a BERT model with additional pre-training on biomedical data and **FLAN-T5-base**⁴ that specializes adapting for several different tasks. The former model outperformed the latter on MedQA by a wide margin so we decided to continue our project only using PubMedBERT.

To widen the variety of our baselines, we have tried adding a pre-training step as a masked language modeling task on MedQA's textbooks, which simply are medical textbooks. This step optimizes all of the model's parameters.

⁴FLAN-T5-base <https://huggingface.co/google/flan-t5-base>

3.3.2 Agent Doctor Responses Evolution

The simulation aspect of Agent Hospital has been simplified to a fine-tuning task because we lack information about the original paper's implementation. Additionally, LLM fine-tuning, in our context, can be seen as the evolution process of a doctor agent meeting new patients everyday. More precisely, the task for the doctor agent is a multiple choice question for each patient it encounters. The doctor has access to the patient's information, such as age, gender, symptoms, travel history in addition to a list of 5 mixed diseases. Then, it must identify the correct one out of the five based on the information provided. Moreover, the fine-tuning technique first used for this process was full fine-tuning to allow the doctor to reach optimal results in the simulation. We have then used partial fine-tuning to make sure the doctor agent retains some of its previous knowledge instead of completely optimizing for the simulation.

4 Evaluation Methods

4.1 Evaluation of Synthetic Profiles

4.1.1 Clinical Expert Evaluation

After generating synthetic patients across two departments, 100 selected profiles were manually validated with the support of an expert. This validation was conducted not only to assess the model's generation capability, but also to verify the soundness of our logic before introducing the profiles into the doctor agent training phase, aiming to minimize potential noise.

4.1.2 Similarity Analysis with Cosine and BERTScore

The evaluation of the match focuses on the generated symptoms in relation to the Full Summary of the disease. The thresholds for these two metrics are set as: *A cosine similarity and BertScore are greater than 0.75*. Only patients meeting both criteria are retained for the final evaluation set.

4.2 Evaluation of Doctor Agent

We always use the accuracy since it reflects really well the performance in a multiple choice answering framework. Indeed, the goal of our medical agent is to have the best possible accuracy on the MedQA test but also the best possible accuracy when diagnosing patients. However, to align with our goal of improving performance on the real-life evaluation of doctor agents, we will compare our

model’s results to our baselines’ results on MedQA and we won’t put as much focus on the virtual evaluation.

5 Experimental Design

5.1 Datasets

- **Collection of Common Diseases** : The common diseases were retrieved from a billing reference list provided by the British Columbia government for [Cardiology](#) and [Infectious and Parasitic](#) diseases.
- **Collection of Disease Details** : The details of each disease were retrieved by communicating with the MedlinePlus API. **MedlinePlus**⁵ is a public resource from the U.S. National Library of Medicine, closely aligned with Baidu’s dataset. An example of the XML returned by this API can be found in Annex A.
- **Collection of Extracted Information**: The collection of disease details can be downloaded from [Cardiology Diseases](#) and [Infectious and Parasitic Diseases](#).
- **Dataset Used for Training the Doctor Agent**: The main datasets are the synthetic patients we generated and the **MedQA**⁶ dataset. MedQA contains textbooks and a set of test data. The textbooks are used for additional pre-training of PubMedBERT, and the test data are used for real-life evaluation of the agent.
- **Synthetic Patient Generated and Modified Datasets**: Our synthetic patient datasets for two departments, Cardiology and Infectious and Parasitic Diseases, can be downloaded here: [Generated Profiles](#) and [Modified Profiles](#).

5.2 Baselines Used for Evaluating the Doctor Agent

Since our goal is not to compete with SoTA results but rather to see whether we can attain better results using alternative techniques, we have built our own baselines and we do not compare our results with published ones. The methods used as baselines are described in section 3.3.1.

⁵MedlinePlus: <https://medlineplus.gov/>

⁶MedQA dataset <https://drive.google.com/file/d/1ImYUSLk9JbgHXOemfvYiDiirluZHPeQw>

5.3 Validation Setup for Synthetic Patient Profiles

- **BERTScore**: We used the model *yikuan8/Clinical-Longformer* (Yikuan Li, 2022) to compute BertScore between generated patient symptoms and disease summaries. This model was chosen because it was pre-trained on over 2 million MIMIC-III clinical notes (Alistair E.W. Johnson, 2016), thus it helps better capture the real-world medical terminology and symptom descriptions. Its ability to handle long input sequences (up to 4 096 tokens) makes it well-suited for our task, where the full summary contains normally more than 512 tokens. Hence, it ensure more accurate and context-aware similarity evaluation.
- **Cosine Similarity**: To conduct the validation using cosine similarity, we first embed both the patient symptoms and the disease summaries into a shared vector space using *Bio_ClinicalBERT* which is a domain-specific BERT model pre-trained on clinical texts. This allows us to assess the semantic alignment between the synthetic patient profiles and expert-written disease descriptions. Semantic similarity is then measured via cosine distance, enabling the filtering of patient profiles that do not align well with the clinical characteristics of the target disease.

6 Experimental Details

This section focuses on the details of the experiments conducted for training the doctor agent.

6.1 Model configurations

For almost all experiments, we used PubMedBERT (PMB) by adding a classifying head for multiple choice question answering.

6.1.1 Hyperparameter settings

The hyper-parameters that were optimized are:

- *Epochs*: 1 to 10
- *Learning rates*: $2e^{-3}$, $2e^{-4}$, $2e^{-5}$

The selection criteria was validation accuracy. The optimal number of epochs varies from test to test but it tends to be around 8. The optimal learning rate for the simulation task is $2e^{-5}$ and $2e^{-4}$ for the real-life evaluation.

6.2 Implementation Details

We can separate our experiments and results into three iterations:

Iteration 1: The two baselines were PMB and PMB+t (PubMedBERT with additional pre-training on textbooks). The simulation task was full fine-tuning on generated patients from *mistral_7B* and *llama2_7B*. These patients all carry diseases from the cardiology department. The resulting model is called *Doctor Agent 1.0*. Finally, these three models were all fully fine-tuned on MedQA for real-life evaluation.

Iteration 2: The only difference from the first iteration here is that the transformer layers of PMB, PMB+t and *Doctor Agent 1.0* were all fine-tuned using LoRA on MedQA instead of full fine-tuning. The classifying head’s fine-tuning remained the same.

Iteration 3: For this iteration, the patients were generated using a stronger model, *llama3_8B* and with more various diseases. Indeed, they carry diseases from the cardiology as well as the infectious and parasitic department. Moreover, the simulation was changed from full fine-tuning to partial fine-tuning. In fact, only the two last transformer blocks as well as the classifier head were optimized. In addition, we built one doctor agent that only encounters original patients (regular patients) during the simulation, and another agent that encounters both original and *augmented patients*. For more information about *augmented patients*, refer to section 3.2.3. Let us refer to these two models as *Doctor Agent 2.1* and *Doctor Agent 2.2* respectively. Similarly to iteration 2, LoRA was used to evaluate these agents on MedQA.

Afterwards, we have decided to also implement a doctor agent with the newly generated patients from *llama3_8B* but keeping the simulation in its previous full fine-tuning form to be able to directly compare the two fine-tuning techniques utilized for simulation specifically. This agent is called *Doctor Agent 2.0*.

Lastly, the final doctor variant is *Doctor Agent 3.0* which is the same as agent 2.0, but simulated on the final set of filtered patients mentioned in section 4.1.2. A recap of every doctor agent implemented is shown in table 1.

7 Results and Analysis

7.1 Generation of Synthetic Patients

Among the three models mentioned in section 3.2.2, *Mistral 7B* produced more natural-sounding language than *LLaMA2 7B*, which performed less ef-

	Generation model	Fine-tuning
Doctor Agent 1.0	Mistral_7B and Llama2_7B	Full
Doctor Agent 2.0	Llama3_8B_Instruct (original)	Full
Doctor Agent 2.1	Llama3_8B_Instruct (original)	Partial
Doctor Agent 2.2	Llama3_8B_Instruct (original + augmented)	Partial
Doctor Agent 3.0	Llama3_8B_Instruct (filtering + augmented)	Full

Table 1: Simulation set-up for every doctor agent implemented.

fectively overall. However, *Llama3 8B - Instruct* demonstrated superior contextual understanding, particularly when interpreting complex summaries of diseases. For example, the case of *Congenital Heart Disease* arises when the fetal heart does not develop normally during pregnancy. Certain maternal risk factors such as rubella, poorly controlled diabetes, or the use of harmful medications in early pregnancy can increase the likelihood of this defect. Both *LLaMA2* and *Mistral 7B* generated patients who were themselves pregnant, failing to capture the correct clinical context. In contrast, *Llama3 8B - Instruct* demonstrated a better understanding by aligning patient profiles more accurately with the intended scenario. Given these observations, *Llama3 8B - Instruct* was selected for the continuation of the work, as it provided the most accurate contextual interpretation and generated clinically coherent patient profiles. The full summary extracted for each disease (see Section 3.2.1) was used as the primary resource for generating patient profiles. Diseases with no identifiable signs or symptoms were excluded from the generation process.

As a result, five synthetic patients were generated per disease, yielding a total of **3404** patients: *1880 in the cardiology category* and *1524 in the infectious and parasitic diseases category*. Each patient profile contains the following fields: *Full Name*, *Age*, *Gender*, *Symptoms*, *Symptom Duration*, *Recent Travel History*, *Medical History*, *Mixed Diseases*[†], *Correct Disease*[†], *Diagnosis Cause*, *Patient Number*. An example profile is provided in the annex C.

7.2 Augmented Synthetic Patient Profiles

The results described in Section 3.2.3 led to the addition of an *altTitle* field in the JSON file (see Section 3.2.1), which replaces the originally gen-

erated disease name. The correct disease name is also embedded within the patient’s profile. As part of the modification, a new field, *altName*[†], is introduced. An example of this modification is provided in Annex D. These modified profiles are presented to the doctor agent during both training and evaluation phases, allowing the agent to learn and generalize new knowledge throughout the simulation task.

All fields marked with[†] are masked during the training of the doctor agents.

7.3 Evaluation Synthetic Patients

7.3.1 With Clinical Expert

The validation process was conducted based on the fields: *age*, *gender*, *symptoms*, *symptom duration*, *recent travel history*, *mixed diseases*, *correct disease*, and *diagnosis cause*. The results of this evaluation are presented below:

Indicator	Percentage	Notes
Fully accepted profiles	90%	Avg score ≥ 1.5
Accepted profiles (minor adjustment)	3%	Avg score ≥ 1.5
Rejected profiles	7%	Avg score < 1.5
* Usable profile rate **		
93% (90+3)		

* Profiles considered usable include both fully accepted and minor-adjusted ones.

** Value expressed as a sum of the two categories.

Table 2: Summary of expert validation results.

This list can be consulted with this [Expert-Annotated Profiles](#). The result of this evaluation identified 3 cases (patients 703, 707, and 708) as partially accepted. In these cases, the diagnosis cause was found to be illogical, even though the correct disease appeared among the provided options. This suggests that the symptoms of the generated disease were very similar to those of another condition. For example, the case of the patient 703, *Vaginosis* would have been a more appropriate diagnosis than *Vaginitis* and the diagnosis cause, and the diagnosis cause should have been *bacterial vaginosis* rather than *candidiasis*.

The seven rejected cases correspond to diseases that are typically asymptomatic such as *aneurysms*. As a result, all the generated patients presented symptoms that do not align with the nature of these diseases, which can be interpreted as model hallucinations. These expert observations suggest that patient generation can fail due to the model’s misunderstanding of similar diseases, outdated source information or uncontrollable hallucinations.

7.3.2 With Cosine Similarity and BERTScore

Following the clinical expert evaluation presented in Section 7.3.1, 25 cases related to the disease *Aneurysms* were excluded, resulting in a refined dataset of 3379 valid cases. Figure 2 presents the outcome of the filtering process, in which unaccepted patients were removed based on validation using BERTScore and cosine similarity. After this step, 153 patients were removed, resulting in a final dataset of 3,251 patients. The filtered patients dataset can be found [here](#).

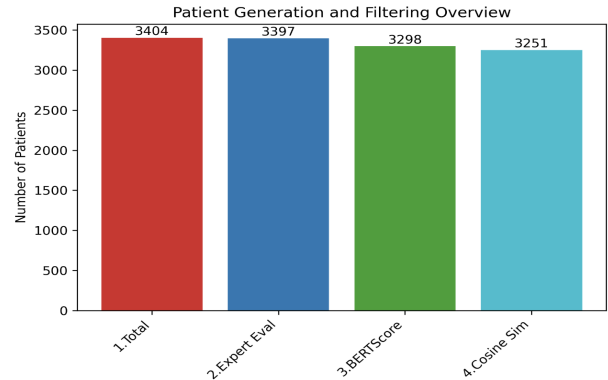


Figure 2: Patient Filtering Overview.

7.4 Real-life Evaluation on MedQA

Iteration 1	Best valid accuracy	Test accuracy
FLAN-T5-base baseline	0.2461	N/A
PMB baseline	0.3357	0.3299
PMB+t baseline	0.3239	0.3543
Doctor Agent 1.0	0.3160	0.3142

Table 3: Real-life evaluation of baselines and the doctor agent on **MedQA test** using **full fine-tuning**.

Iterations 2 & 3	Best valid accuracy	Test accuracy
PMB baseline	0.3129	0.32.05
PMB+t baseline	0.3011	0.3126
Doctor Agent 1.0	0.3066	0.3284
Doctor Agent 2.0	0.2940	0.3001
Doctor Agent 2.1	0.2940	0.3048
Doctor Agent 2.2	0.2838	0.3001
Doctor Agent 3.0	0.3003	0.3040

Table 4: Real-life evaluation of baselines and the doctor agent on **MedQA test** using **LoRA**.

The results from the first iteration show that our first attempt at improving the doctor agent has failed since the performance of Doctor Agent 1.0 is worse than the performance of the two baselines PMB and PMB-t. Some potential issues were the lack of patient diversity and the fine-tuning

techniques used not only during simulation but also for real-life evaluation. The former problem makes it so the simulation task encourages the model to memorize the specific cardiological diseases and their symptoms for example, instead of enhancing its medical knowledge and thinking abilities. The latter problem leads to catastrophic forgetting since using full fine-tuning changes every parameters of the model to optimize according to the given task. This prevents the doctor agent from keeping the key medical and language knowledge PubMedBERT has.

Moving on the iterations 2 and 3, we can observe that the overall accuracies are lower than the ones for iteration 1. That was to be expected because using LoRA for fine-tuning on the evaluation task does not allow as much optimization as using full fine-tuning. The advantages it brings are that fine-tuning is lighter to execute and that the resulting models still use in some way the parameters they had previously learnt.

The best test accuracy is achieved by the same doctor agent that was trained for iteration 1 but using LoRA for the evaluation on MedQA instead of full fine-tuning. This could be interpreted as the achievement of our goal, but the accuracy difference is rather small and our other results contradict with this one. Consequently, it could also be interpreted as a fluke.

Furthermore, all other doctor agents under-perform since their validation and test accuracies are lower than the ones achieved by the baselines and by Doctor Agent 1.0. The cause for this could be the use of Llama3_8B instead of Llama2_7B and Mistral_7B for patient generation or it could be the fact that generated patients come from 2 different medical departments instead of 1. We originally thought that these two changes would be benefic by orienting the simulation task towards general knowledge rather than information memorization but it is not the case. In addition, the best validation accuracy remains the simple PubMedBERT baseline.

Now looking at the results of Doctor Agents 2.0, 2.1 and 2.2 specifically, we see that all 3 models yield very similar results. The best out of the bunch is the version 2.1 which uses partial fine-tuning during simulation and only uses

original patients (no augmented patients). The version 2.0 is very close behind so we conclude that whether full or partial fine-tuning is used for simulation does not affect the output to a certain degree. Finally, the model 2.2 being the worst indicates that our regularization method of including augmented patients during simulation decreases the performance for real-life evaluation.

Additionally, the results of agent 3.0 are slightly better than the ones of agent 2.0. This reveals that the filtering of generated patients with clinical expert evaluation and similarity analysis effectively improves the performance of the doctor agent. Nonetheless, the upgrade is not sufficient to overcome the baselines and agent 1.0.

Lastly, we observed that, in Doctor Agent 1.0, approximately 5000 patients were generated using two models for a single department (cardiology). Regardless of the generated model’s performance, the resulting data still proved useful for training the doctor agent. In contrast, Doctor Agent 3.0 involved only about 3000 patients, generated using one model across two departments, yet yielded slightly better results than Agent 2.0. This suggests that even with fewer patients per department, the manual and dynamic filtering of illogical cases plays a crucial role. If possible, generating a larger pool of patients and then applying stricter evaluation metrics may be a more effective strategy for improving overall performance.

7.4.1 Virtual Evaluation

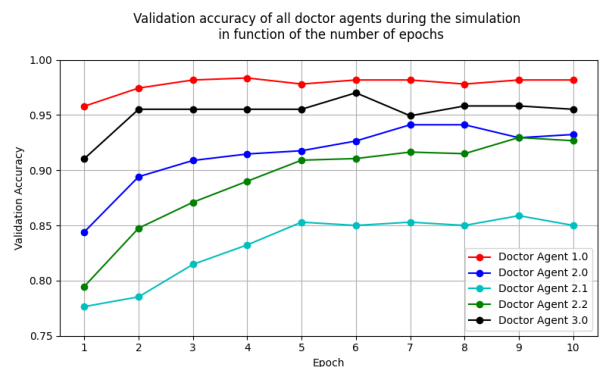


Figure 3: Validation accuracy of the different doctor agents during the simulation.

First of all, we see in figure 3 that every doctor agent improved on the simulation task with the right amount of training. This is essential as it en-

capsulated the evolution of the agents. Next, the figure shows that the Doctor Agent 1.0 performs better in simulation than the other agents, which is to be expected since the generated patients for it only come from the cardiology department. The four other agents interact with patients from 2 different medical departments which adds variety and complicates the virtual task. Next, as we also expected, agent 2.0 yields better accuracies than agents 2.1 and 2.2. That is because the former model was fully fine-tuned on the simulation task compared to the latter models which were only partially fine-tuned. This fine-tuning constraint restrained the performance of these models. Then, the fact that agent 2.2 outperforms agent 2.1 indicates that the simulation is easier when the doctor agent encounters original and augmented patients rather than only encountering original patients. Normally, it would be the opposite because agent 2.2 has the same job as agent 2.1 with the additional underlying task of recognizing equivalent diagnoses. One possibility that could explain this phenomena is if the augmented patients are too similar to the original ones. Indeed, half of the patients would then be easier to treat when having seen the other half that resembles it beforehand. Lastly, the validation accuracy of agent 3.0 being significantly higher than the one of agent 2.0 implies that the filtering of problematic patients has made the simulation easier for the model to generalize.

8 Conclusion

In the end, we have managed to implement, from scratch, an LLM training pipeline similar to the one described in *Agent Hospital* for the development of doctor agents. It consists of a patient generation step, a simulation step and an evaluation step. We also have been able to test it on lower scale AI models, namely BERT and T5. Our objective of increasing a doctor agent’s accuracy on MedQA by using the simulation training we have implemented has been achieved in an isolated case. Indeed, it was done by using PubMedBERT as the base model, then proceeding to extra pre-training as masked language modeling on medical textbooks and finally running a simulation in the form of full fine-tuning for the doctor agent to diagnose patients that were generated by Mistral_7B, Llama2_7B, and Llama3_8B. Furthermore, the evaluation of that agent on MedQA used LoRA. This procedure led to the

best test accuracy obtained on the medical test by utilizing LoRA. However, the difference in performance is not significant enough and our results are too inconsistent for us to claim that our methodology leads to an improvement of low scale doctor agents. And we have not yet explored all the options possible so we cannot claim that a simulation environment cannot be useful for a modest LLM doctor agent. In order to get closer to that answer, we propose the following directions:

- Replace fine-tuning with Information-Retrieval (IR) to simulate reasoning over external knowledge base.
- Explore decoder-only or encoder-decoder architectures such as *T5 variants* pretrained on medical data.
- Expand the scope of generated patients to cover other medical departments.
- Investigate advanced fine-tuning technique such as *LoRA+* which may improve convergence speed and stability.
- Use (symptom $\rightarrow$ MeSH ⁷ code) pairs as an additional data source for patient generation since the MedlinePlus dataset is, in some cases, outdated. This change can help to internalize up-to-date and structured medical knowledge, therefore enhancing the reasoning ability and conceptual understanding of the patient generator agent.

9 Contributions of Each Team Member

Task	Assignee
Patient Agent Generation	Nhung
Doctor Agent Baselines	Philippe
Patient Agent Generation Evaluation	Nhung
Patient Agent Modification	Nhung
Complexify Doctor Agent Baselines	Philippe
Doctor Agent Response Evolution	Philippe

Table 5: Tasks and Assignment

References

Arthur Mensch Chris Bamford Devendra Singh Chaplot Diego de las Casas Florian Bressand Gianna Lengyel Guillaume Lample Lucile Saulnier L  lio Renard Lavaud Marie-Anne Lachaux Pierre Stock Teven Le Scao Thibaut Lavril Thomas Wang Timoth  e Lacroix William El Sayed Albert Q. Jiang,

⁷MeSH <https://www.ncbi.nlm.nih.gov/mesh>

Alexandre Sablayrolles. 2023. [Mistral 7b](#). *Preprint*, arXiv:2310.06825v1.

* Lu Shen2 Li-wei H. Lehman1 Mengling Feng1 Mohammad Ghassemi1 Benjamin Moody1 Peter Szolovits4 Leo Anthony Celi1 2 Roger G. Mark1 Alistair E.W. Johnson, Tom J. Pollard1. 2016. [Data descriptor: Mimic-iii, a freely accessible critical care database](#). *Preprint*, nature:sdata201635.

Yu Gu, Robert Tinn, Hao Cheng, Michael Lucas, Naoto Usuyama, Xiaodong Liu, Tristan Naumann, Jianfeng Gao, and Hoifung Poon. 2021. [Domain-specific language model pretraining for biomedical natural language processing](#). *ACM Transactions on Computing for Healthcare*, 3(1):1–23.

Kevin Stone† Peter Albert Amjad Almahairi-Yasmine Babaei Nikolay Bashlykov Soumya Batra Prajjwal Bhargava Shruti Bhosale Dan Bikel Lukas Blecher Cristian Canton Ferrer Moya Chen Hugo Touvron, Louis Martin†. 2023. [Llama 2: Open foundation and fine-tuned chat models](#). *Preprint*, arXiv:2307.09288v2.

Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. 2020. [What disease does this patient have? a large-scale open domain question answering dataset from medical exams](#). *Preprint*, arXiv:2009.13081.

Junkai Li, Yunghwei Lai, Weitao Li, Jingyi Ren, Meng Zhang, Xinhui Kang, Siyu Wang, Peng Li, Ya-Qin Zhang, Weizhi Ma, and Yang Liu. 2025. [Agent hospital: A simulacrum of hospital with evolvable medical agents](#). *Preprint*, arXiv:2405.02957.

AI @ Meta Llama Team. 2024. [The llama 3 herd of models](#). *Preprint*, arXiv:2407.21783v3.

Faraz S. Ahmad Hanyin Wang-Yuan Luo Yikuan Li, Ramsey M. Wehbe. 2022. [Clinical-longformer and clinical-bigbird: Transformers for long clinical sequences](#). *Preprint*, arXiv:2201.11838v3.

Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2020. [Bertscore: Evaluating text generation with bert](#). *Preprint*, arXiv:1904.09675v3.

A Appendix: Collection of Disease Details

With the query to the API [here](#), the following result was returned.



Figure 4: Example of XML response returned by the MedlinePlus API for the query "Chronic Hypotension".

B Appendix: Full Disease Summary Example

This appendix shows a sample of a full disease summary used for patient generation.

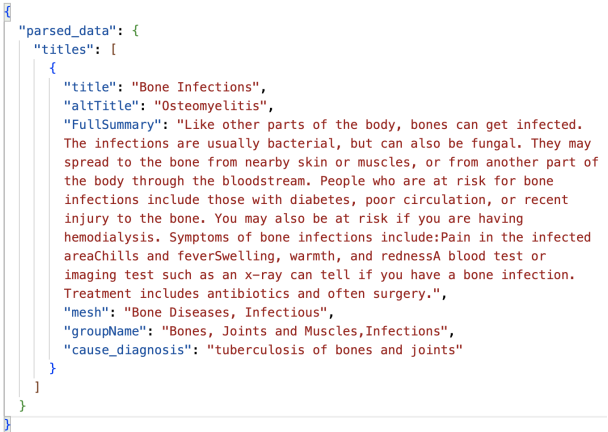


Figure 5: Example of a parsed full disease summary for "Bone Infections" extracted from MedlinePlus, used for synthetic patient generation.

C Appendix: Generated Patient Exmple

This appendix shows an example of a generated patient with "Heart Valve Diseases".



Figure 6: Example of a synthetic patient with Heart Valve Diseases.

D Appendix: Modified Patient Profile Example

This appendix shows an example of a generated patient profile for "Heart Valve Diseases" after modification.

```
{
  "Full name": "Maria Sanchez",
  "Age": "38",
  "Gender": "Female",
  "Symptoms": "Dizziness, fainting episodes",
  "Symptom duration": "A few months",
  "Recent travel history": "None",
  "Medical history": "Congenital heart defect, now asymptomatic",
  "Mixed diseases": [
    "Hand Injuries and Disorders",
    "Valvular heart disease",
    "Cholesterol",
    "Heart Disease in Women",
    "Blood Clots"
  ],
  "Correct disease": "Valvular heart disease",
  "Diagnosis cause": "acute rheumatic fever",
  "patient_number": "0001868",
  "altName": [
    "Heart Valve Diseases",
    "Aortic Stenosis",
    "Heart Murmur",
    "Valvular Heart Diseases",
    "Valvular heart disease"
  ]
},
```

Figure 7: Example of a generated patient profile for "Heart Valve Diseases" after modification.